

Kubernetes

What is Kubernetes?

Kubernetes (K8s) is a free, open-source tool that helps you manage and control applications running inside containers.

First, understand containers:

- A container is a small package that contains an application and everything it needs to run (code, libraries, settings).
- For example, a website might run inside a container.

When you have only one container, managing it is easy. But imagine a big company with:

- 100 containers
- Running on 20 different computers (servers)
- Thousands of users accessing the application

Managing all of them manually becomes difficult. This is where Kubernetes helps.

Why do we need Kubernetes?

Kubernetes automatically handles tasks like:

✅ Starting containers

If an application needs to run, Kubernetes starts the required containers.

✅ Stopping and restarting containers

If a container crashes, Kubernetes notices and restarts it automatically.

✅ Scaling applications

If many users come to your website, Kubernetes can create more containers to handle the traffic.

Example:

Normal day:

Website

|

2 containers

Busy day:

Website

|

10 containers

Kubernetes can increase or decrease containers automatically.

✓ Load balancing

If you have many containers, Kubernetes distributes users between them so one container does not get overloaded.

What does "orchestration" mean?

Orchestration means organizing and managing many things together.

Think about an orchestra:



- Many musicians
- Different instruments

- One conductor controlling everything

Kubernetes is like the conductor:

- Containers = musicians
- Servers = stage
- Kubernetes = conductor

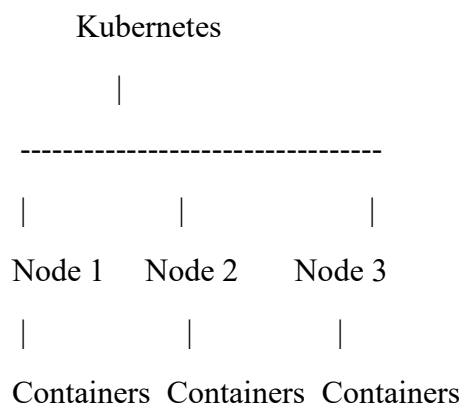
It makes sure everything works together.

What are nodes?

A node is a machine (computer/server) where containers run.

Example:

Kubernetes Cluster



A group of nodes managed by Kubernetes is called a cluster.

What is K8s?

Kubernetes has 10 letters:

K U B E R N E T E S

People shorten it as:

K + 8 letters + S

= K8s

Who created Kubernetes?

- Originally created by Google
 - Based on Google's internal system called Borg
 - Later donated to the Cloud Native Computing Foundation (CNCF), which now maintains it.
-

Simple real-world example

Imagine you run Netflix:

Tell Kubernetes:

"I need my app running with 50 containers"

Without Kubernetes:

- You manually start servers
- Restart failed applications
- Add more servers during high traffic

With Kubernetes:

- Creates them
 - Places them on machines
 - Restarts failures
 - Handles traffic
 - Adds more when needed
-

In one sentence:

Kubernetes is a tool that automatically deploys, manages, scales, and keeps container-based applications running smoothly across many computers.

Features of Kubernetes (K8s)

1. Container Orchestration

- Kubernetes manages many containers.
- It runs containers on different machines (nodes).
- It helps in deployment, scaling, and managing applications.

Example:

Many containers → Kubernetes manages them together.

2. Restart and Recreate Containers

- If a container fails, Kubernetes detects it.
- It automatically restarts or replaces the container.
- Keeps the application running.

Example:

Container ❌ → Kubernetes → New Container ✅

3. Auto-Scaling

- Kubernetes can increase or decrease containers automatically.
- It depends on user traffic.

Example:

Low traffic → 2 containers

High traffic → 10 containers

4. Load Balancing

- Kubernetes distributes users' requests between containers.
- Prevents one container from becoming overloaded.

Example:

Users → Kubernetes → Container 1

→ Container 2

→ Container 3

5. Rolling Updates and Rollbacks

- Updates applications without stopping them.

- If an update fails, it can return to the old version.

Example:

Old Version  → Update → New Version 
↓
Rollback

6. Networking

- Containers can communicate with each other.
 - Kubernetes gives each container a unique IP address.
 - Manages communication between containers.
-

7. Multi-Cloud Support

- Kubernetes can run applications on different clouds.
- Works with cloud and company servers.

Example:

AWS + Google Cloud + Local Server
↓
Kubernetes

Short Note:

Kubernetes helps us to deploy, manage, scale, and run container applications automatically and reliably.

Kubernetes Terminology

1. Pod

A Pod is the smallest unit in Kubernetes.

- A pod contains one or more containers.
- Containers inside a pod share storage and network.
- Application actually runs inside pods.

Example:

Pod

|

|-- Container

|-- Container

Think of a pod as a box that holds containers.

2. Node

A Node is a machine where Kubernetes runs applications.

It can be:

- Physical computer
- Virtual machine

A node runs pods.

Example:

Node 1

|

|-- Pod

|-- Pod

3. Cluster

A Cluster is a group of nodes working together.

Example:

Kubernetes Cluster

Node 1

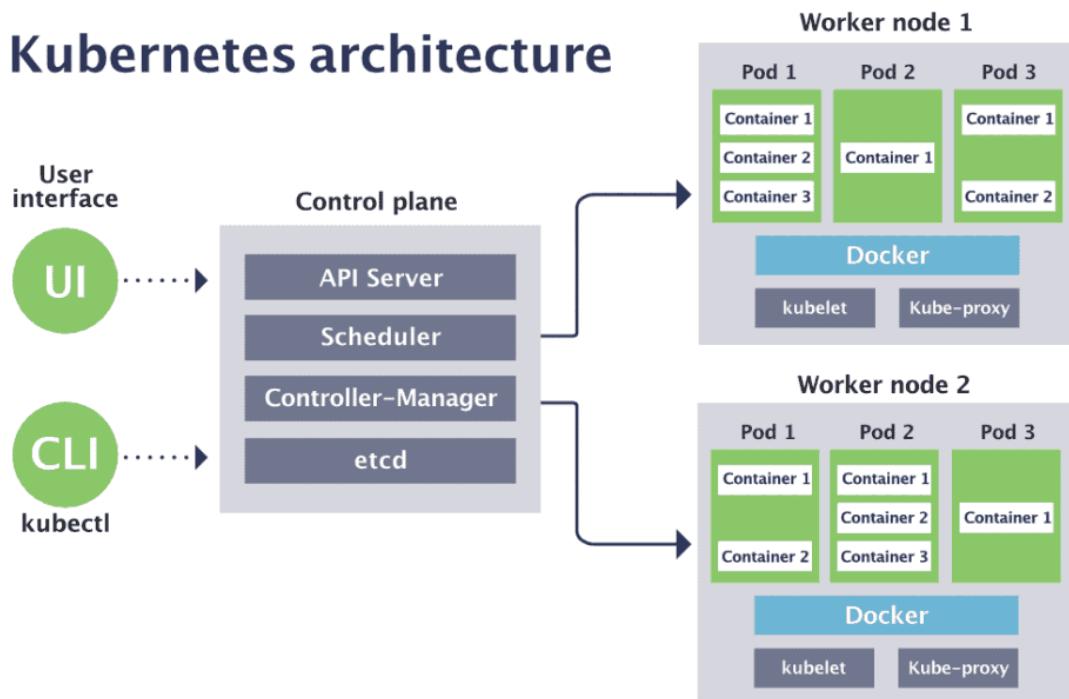
Node 2

Node 3

All nodes together form a cluster.

Kubernetes Architecture

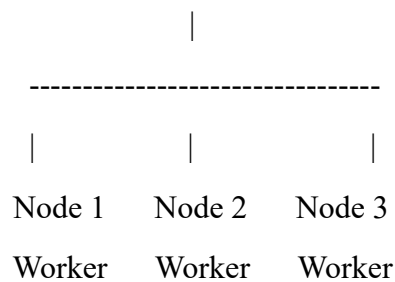
Kubernetes architecture



Kubernetes has two main parts:

Kubernetes Cluster

Control Plane



1. Control Plane (Master)

The Control Plane manages the entire Kubernetes cluster.

It makes decisions like:

- Where to run pods
- How to maintain applications
- How to recover from failures

It contains:

a. kube-apiserver

The API Server is the communication center of Kubernetes.

It:

- Receives requests from users
- Communicates with other components
- Provides access to the cluster

Example:

User → API Server → Kubernetes

b. etcd

etcd is Kubernetes' database.

It stores important cluster information:

- Nodes information
- Pods information
- Configuration
- Current state

Example:

If Kubernetes needs to know:

"Which pods are running?"

It checks etcd.

c. kube-scheduler

The scheduler decides:

"On which node should a pod run?"

It checks:

- CPU availability
- Memory
- Node conditions

Example:

New Pod
|
Scheduler
|
Node 2 (selected)

d. kube-controller-manager

Controller manager keeps the cluster in the desired state.

Example:

You want:

5 Pods running

But one pod crashes:

4 Pods running

Controller manager creates a new pod.

It continuously checks and fixes problems.

e. cloud-controller-manager

(Optional)

Used when Kubernetes runs on cloud platforms.

Examples:

- AWS
- Google Cloud
- Azure

It manages:

- Cloud load balancers
- Storage
- Cloud networking

2. Worker Node

Worker nodes are the machines where applications actually run.

Each worker node contains:

Worker Node

- kubelet
 - kube-proxy
 - Container Runtime
 - Pods
-

a. kubelet

kubelet is the worker node agent.

It:

- Receives instructions from API server
- Starts containers
- Checks pod health
- Reports status back

Example:

Control Plane:

"Run this pod"

kubelet:

"Okay, I will start it"

b. kube-proxy

kube-proxy manages networking.

It:

- Connects users to pods
- Routes traffic
- Maintains network rules

Example:

User

|

kube-proxy

|

Pod

c. Container Runtime

Container runtime runs containers.

Examples:

- Docker
- containerd

It:

- Downloads container images
 - Starts containers
 - Stops containers
-

d. Pods

Pods are where applications run.

Example:

Worker Node

|

Pod

|

Container

kubelet ensures pods are healthy.

Interview Short Answer:

"Kubernetes architecture consists of a Control Plane that manages the cluster and Worker Nodes where applications run. The Control Plane contains API Server, etcd, Scheduler, and Controllers. Worker Nodes contain kubelet, kube-proxy, container runtime, and pods."